Asymptotic notation

Week #4 - KIT205 - Data Structures and Algorithms

Asymptotic notation

Describes how an algorithm's resource usage (time or space) scales as the input grows.

Focus on the shape of growth: how resource usage changes relative to input size, not exact runtimes.

Offers a universal description of growth behavior, independent of specific measurements (e.g., "3 seconds on my laptop").

$f(n)$

Growth is expressed in terms of a function $f(n)$, where n is the input size.

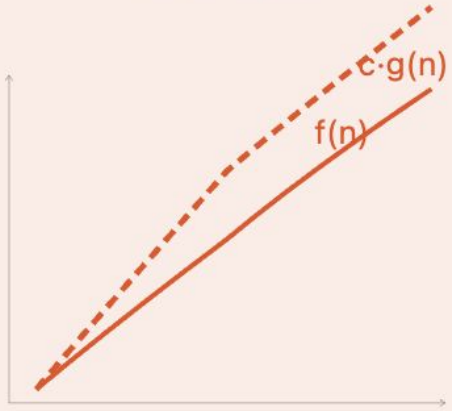
Asymptotic notation - Three Notations

Big O O(g)

Upper bound

$f(n) \leq c \cdot g(n)$

for all $n \geq n_0$



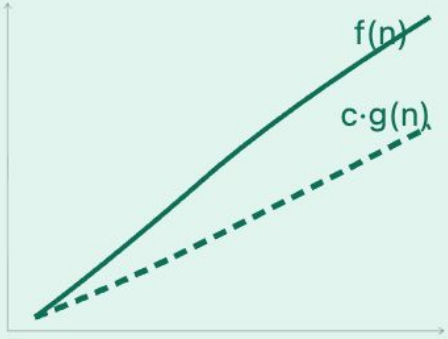
"at most this fast"

Big Ω Ω(g)

Lower bound

$f(n) \geq c \cdot g(n)$

for all $n \geq n_0$



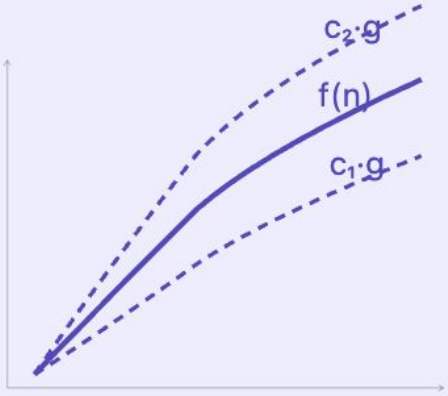
"at least this fast"

Big Θ Θ(g)

Tight bound

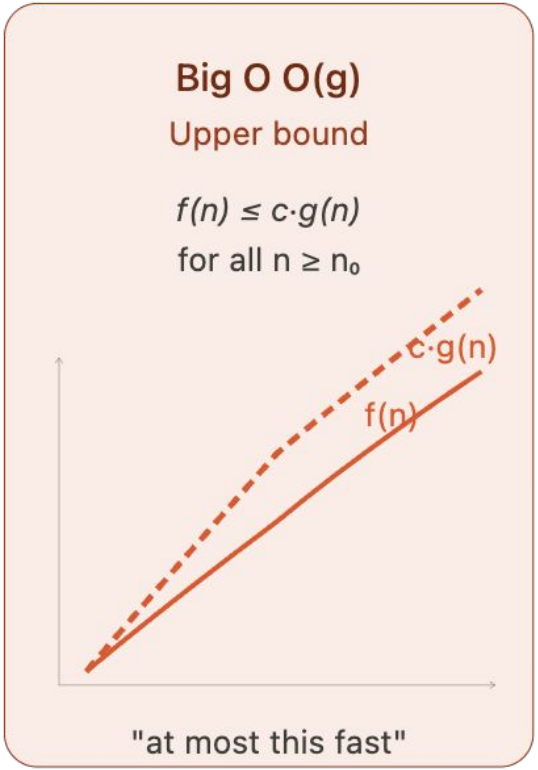
$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$

for all $n \geq n_0$



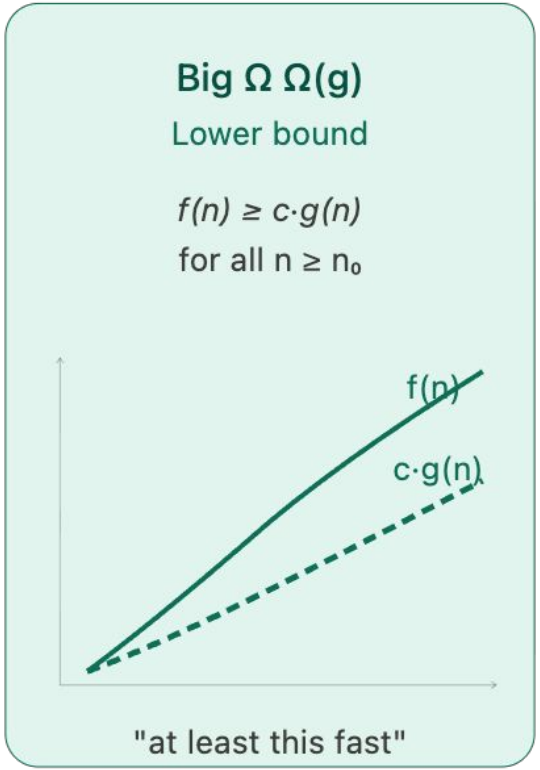
"exactly this fast"

Asymptotic notation - Three Notations



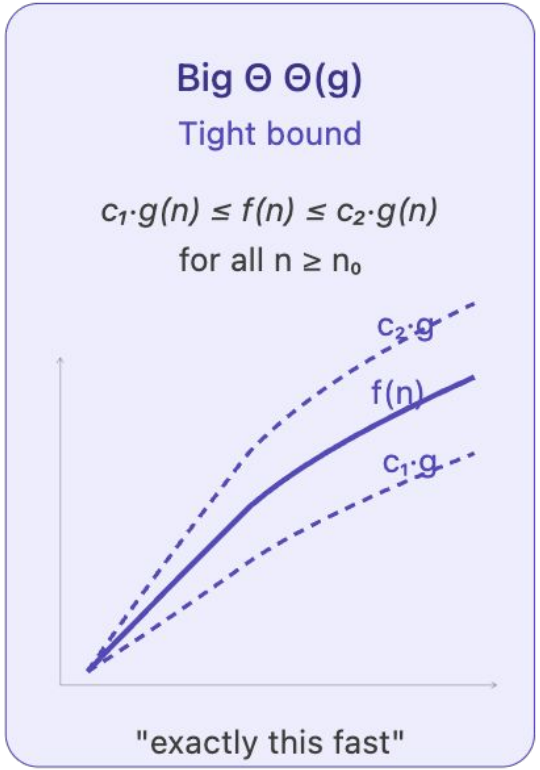
- **Big O → The Ceiling**
  - *f(n) = O(g(n))* means that f grows no faster than *g*.
  - It represents the worst-case guarantee for an algorithm.
  - Saying "this algorithm is $O(n^2)$" means its growth rate will not exceed n squared, no matter how bad the input is.
  - Technically, $O(n^2)$ is also $O(n^3)$ (any upper bound works), but the tightest bound is the most useful.

Asymptotic notation - Three Notations



- **Big Omega → the lower bound:**
  - *f(n) = Ω(g(n))* means f grows no slower than *g.*
  - It provides the best-case guarantee.
  - Saying an algorithm is *Ω(n)* means it must do at least linear work (you can't do better than that for this problem).

Asymptotic notation - Three Notations



- **Big Theta** → The "sandwich" bound.
  - *f(n) = Θ(g(n))* means *f* is squeesed between two constants times *g*.
  - It represents the *tight bound* which is the most precise description of an algorithm's running time.
  - If an algorithm is *Θ(n log n),* it is simultaneously *O(n log n)*(upper bound) and *Ω(n log n)* (lower bound).

Asymptotic notation - Common algorithmic complexities

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$

Best

$O(1)$

Hash lookup, array index

Great

$O(\log n)$

Binary search, balanced BST

Good

$O(n)$

Linear scan, finding max

Acceptable

$O(n \log n)$

Merge sort, heap sort

Caution

$O(n^2)$

Bubble sort, nested loops

Avoid

$O(2^n)$

Naïve recursion, subsets

Asymptotic notation - Systematic rules

Simple statements are O(1)

Any single operation; an assignment, arithmetic, comparison, array access, `return`, costs constant time regardless of input size.
The number of *n*-independent operations is irrelevant; even 1,000 consecutive statements is still O(1).

```
int x = arr[0];     // O(1)
x = x * 2 + 5;     // O(1)
if (x > 10) ...    // O(1)
```

Asymptotic notation - Systematic rules

A single loop is O(n)

A loop that runs proportional to the input size *n* costs O(n). The key is: does the loop bound depend on *n*?

O(n) — single loop

```
for (int i = 0; i < n; i++) {
    sum += arr[i];    ← O(1) body
}
Total: n × O(1) = O(n)
```

O(n) — while loop variant

```
int i = 0;
while (i < n) {
    process(arr[i]);  ← O(1)
    i++;
}
Same shape, same cost: O(n)
```

O(1) — fixed-bound loop

```
for (int i = 0; i < 1000; i++) {
    x += i;
}
Bound is a constant, not n → O(1)
```

O(n/2) = O(n) — partial loop

```
for (int i = 0; i < n/2; i++) {
    swap(arr[i], arr[n-1-i]);
}
n/2 iterations — drop constant → O(n)
```

Asymptotic notation - Systematic rules

Sequential blocks add (then take the dominant term)

When blocks of code follow each other (not nested), you add their costs and then drop the lower-order term.
$O(f) + O(g) = O(\max(f, g))$ when f and g are comparable growth rates.

```
// Block A: Iterating an array → O(n)
for (int i = 0; i < n; i++) { sum += arr[i]; }

// Block B: nested sort → O(n²)
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
        if (arr[i] > arr[j]) swap(&arr[i], &arr[j]);

// Total: O(n) + O(n²) = O(n²)  ← O(n) vanishes
```

Asymptotic notation - Systematic rules

Logarithmic loops (halving/doubling)

When the loop variable is multiplied or divided each step (rather than incremented), it costs O(log n).

O(log n) — dividing

```
for (int i = n; i > 1; i = i / 2) {
    printf("%d\n", i);
}
```

n=16: i = 16 → 8 → 4 → 2 → stop (4 steps = $\log_2 16$)

Each step halves i → runs $\log_2(n)$ times → **O(log n)**

O(log n) — multiplying

```
for (int i = 1; i < n; i = i * 2) {
    process(i);
}
```

n=16: i = 1 → 2 → 4 → 8 → stop (4 steps)

Each step doubles i → $\log_2(n)$ iterations → **O(log n)**

O(n log n) — loop inside a halving loop

```
for (int i = 1; i < n; i *= 2) {    ← log n times
    for (int j = 0; j < n; j++) { ← n times each
        work(i, j);
    }
}
```

log n × n = **O(n log n)** — the shape of merge sort

Asymptotic notation - Systematic rules

Function calls: substitute their complexity

When a function call appears inside a loop, you replace it with the function's own complexity and then multiply.
If the called function is *O(1)* (e.g. `push()` on a stack), then the outer loop stays *O(n)*. Always check what the function does, not just that it's called.

```
void printAll(int arr[], int n) {   // O(n)
    for (int i = 0; i < n; i++)
        printf("%d", arr[i]);
}

// Calling it inside a loop:
for (int i = 0; i < n; i++) {      // n times...
    printAll(arr, n);              // ...each call costs O(n)
}
// Total: O(n) × O(n) = O(n²)
```

Asymptotic notation - Systematic rules

Recursion: count the call tree

Recursive functions need you to reason about how many times the function is called and what work is done each call. The two most common patterns:

O(n log n) — divide and conquer

```
void mergeSort(int arr[], int lo, int hi) {
    if (lo >= hi) return;
    int mid = (lo + hi) / 2;
    mergeSort(arr, lo, mid);        ← left half
    mergeSort(arr, mid+1, hi);      ← right half
    merge(arr, lo, mid, hi);        ← O(n) merge
}
```

log n levels of recursion × O(n) merge work per level → **O(n log n)**

O(n) — linear recursion

```
int sum(int arr[], int n) {
    if (n == 0) return 0;                ← base case
    return arr[n-1] + sum(arr, n-1);  ← 1 call, n shrinks by 1
}
```

Call depth = n, O(1) work per call → **O(n)**

O(log n) — halving recursion

```
int binarySearch(int arr[], int lo, int hi, int x) {
    if (lo > hi) return -1;
    int mid = (lo + hi) / 2;
    if (arr[mid] == x) return mid;
    if (arr[mid] < x)
        return binarySearch(arr, mid+1, hi, x); ← half
    return binarySearch(arr, lo, mid-1, x);    ← half
}
```

Problem halves each call → $\log_2(n)$ depth, O(1) work → **O(log n)**

$O(2^n)$ — exponential recursion

```
int fib(int n) {
    if (n <= 1) return n;
    return fib(n-1) + fib(n-2); ← 2 calls each time
}
```

Binary call tree with depth n → roughly 2^n nodes → **$O(2^n)$** — the classic trap

Asymptotic notation - Systematic rules

Recursion: count the call tree

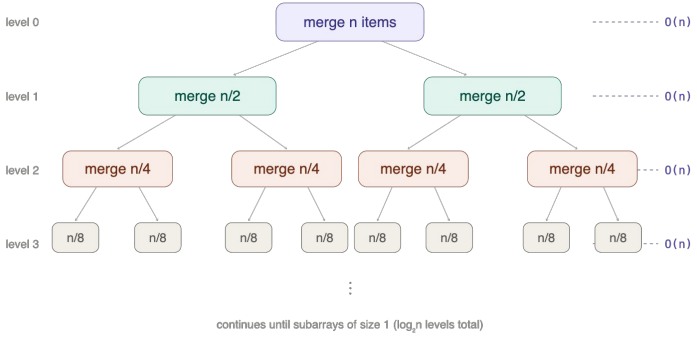
Recursive functions need you to reason about how many times the function is called and what work is done each call. The two most common patterns:

O(n log n) — divide and conquer

```
void mergeSort(int arr[], int lo, int hi) {
    if (lo >= hi) return;
    int mid = (lo + hi) / 2;
    mergeSort(arr, lo, mid);        ← left half
    mergeSort(arr, mid+1, hi);      ← right half
    merge(arr, lo, mid, hi);        ← O(n) merge
}
```

log n levels of recursion × O(n) merge work per level → **O(n log n)**

Level	Number of calls	Size of each subarray	Merge work per call	Total work at this level
0 (root)	1	n	O(n)	O(n)
1	2	n/2	O(n/2) each	2 × n/2 = O(n)
2	4	n/4	O(n/4) each	4 × n/4 = O(n)
3	8	n/8	O(n/8) each	8 × n/8 = O(n)
⋮	2^k	n/2^k	O(n/2^k) each	2^k × n/2^k = O(n)
$\log_2 n$ (leaves)	n	1	O(1) — base case	O(n)
			Grand total (sum all levels)	O(n) × $\log_2 n$ = O(n log n)



The pattern that makes it n log n:
At every level, the calls multiply by 2 but each merge shrinks by half — so the cancels perfectly: `2^k × n/2^k = n` always. Every level does exactly O(n) total work. Since there are `$\log_2 n$` levels, the grand total is `n × $\log_2 n$` = **O(n log n)**.

Asymptotic notation - Cheat Sheet

Step 1 — identify the input
What is n? (array size, string length...)

↓

Step 2 — label each block
Single stmt=O(1), loop=O(n), nested=O(n^2)

↓

Step 3 — expand function calls
Replace each call with its own O(...)

↓

Step 4 — combine
Nested=multiply, sequential=add

↓

Step 5 — simplify
Drop constants, keep dominant term

↓

Result: state as O(f(n))

Quick reference

Pattern	Complexity
Single statement	O(1)
Loop 0..n	O(n)
Loop 0..n/2	O(n)
Loop i*=2	O(log n)
Two nested loops	O(n^2)
Three nested loops	O(n^3)
Log loop + inner n	O(n log n)
Linear recursion	O(n)
Halving recursion	O(log n)
Two-branch recursion	O(2^n)
Divide + O(n) merge	O(n log n)

Seq: O(n) + O(n^2)	→ O(n^2)
Nested: O(n) × O(n)	→ O(n^2)
Drop: O(3n + 12)	→ O(n)
Drop: O(n^2+n)	→ O(n^2)

Asymptotic notation - Putting it all together

```
void analyse(int arr[], int n) {
    int max = arr[0];                        // O(1)

    for (int i = 1; i < n; i++)             // O(n)
        if (arr[i] > max) max = arr[i];     //   O(1)

    for (int i = 0; i < n; i++) {           // O(n)
        for (int j = i+1; j < n; j++) {  //   O(n)
            if (arr[i] + arr[j] == max)     //     O(1)
                printf("found\n");
        }
    }

    for (int i = 1; i < n; i *= 2)          // O(log n)
        printf("%d\n", arr[i]);              //   O(1)
}
```

$O(1) + O(n) + O(n^2) + O(\log n) = O(n^2)$ — the dominant term wins.

Asymptotic notation - Examples

```
int findMax(int arr[], int n) {
    int max = arr[0];          // ①
    for (int i = 1; i < n; i++) { // ②
        if (arr[i] > max)       // ③
            max = arr[i];
    }
    return max;               // ④
}
```

Line ① and ④ are O(1). The loop ② runs exactly n−1 times, each iteration doing O(1) work. Total: O(1) + O(n) + O(1) = O(n).

Asymptotic notation - Examples

```
void printHeader(int arr[], int n) {
    for (int i = 0; i < 10; i++) { // always 10
        printf("%d ", arr[i]);
    }
    printf("\narray has %d items\n", n);
}
```

The loop always runs exactly 10 times regardless of n. Constants collapse to O(1). The printf at the end is also O(1). Total: O(1).

Asymptotic notation - Examples

```
void process(int arr[], int n) {
    for (int i = 0; i < n; i++)   // ← Block A
        arr[i] = arr[i] * 2;

    for (int i = 0; i < n; i++)   // ← Block B
        printf("%d\n", arr[i]);
}
```

Block A = O(n), Block B = O(n). Total = O(n) + O(n) = O(2n). Drop the constant coefficient → O(n).
Two passes is still linear.

Asymptotic notation - Examples

```
void printPairs(int arr[], int n) {
    for (int i = 0; i < n; i++) {      // outer
        for (int j = i; j < n; j++) {  // inner: n-i steps
            printf("%d,%d\n", arr[i], arr[j]);
        }
    }
}
```

Total iterations = $n + (n-1) + (n-2) + \ldots + 1 = n(n+1)/2$. Drop the constant ½ and lower-order terms → $O(n^2)$.

Asymptotic notation - Examples

```
void logSteps(int n) {
    int i = n;
    while (i > 1) {
        printf("%d\n", i);
        i = i / 2;        // halved each step
    }
}
```

Starting at n, dividing by 2 each step: n → n/2 → n/4 → … → 1. That takes $\log_2(n)$ steps. Each step is O(1). Total: O(log n).

Asymptotic notation - Examples

```
void linearSearch(int val, int n) {
    // search() scans a linked list of size n → O(n)
    for (int i = 0; i < n; i++) {
        if (search(val + i, n))   // ← O(n) call
            printf("found\n");
    }
}
```

The outer loop runs n times. Each iteration calls search() which costs O(n). Multiply: O(n) × O(n) = $O(n^2)$.

Asymptotic notation - Examples

```
void mixed(int arr[], int n) {
    int x = arr[0] + arr[n-1];  // Block A

    for (int i = 1; i < n; i *= 2)  // Block B
        printf("%d\n", arr[i]);

    for (int i = 0; i < n; i++)     // Block C
        for (int j = 0; j < n; j++)
            arr[i] += arr[j];
}
```

Block A = O(1). Block B = O(log n) (i doubles). Block C = $O(n^2)$ (nested). Total = O(1) + O(log n) + $O(n^2)$ = $O(n^2)$ ← dominant term wins.

Asymptotic notation - Examples

```
int sumList(Node* head) {
    if (head == NULL)           // base case
        return 0;
    return head->val            // O(1) work
           + sumList(head->next); // recurse on n-1
}
```

The function is called once per node. A list of n nodes produces n+1 calls (including the NULL base case). Each call does O(1) work. Total: O(n).

Asymptotic notation - Examples

```
void divSort(int arr[], int lo, int hi) {
    if (lo >= hi) return;
    int mid = (lo + hi) / 2;
    divSort(arr, lo, mid);       // left half
    divSort(arr, mid+1, hi);     // right half
    merge(arr, lo, mid, hi);     // merge → O(n)
}
```

There are log n levels of recursion (halving each time). At every level, the merge calls collectively process all n elements → O(n) work per level. Total: log n levels × O(n) = O(n log n).

Asymptotic notation - Examples

```
void funky(int arr[], int n) {
    for (int i = 0; i < n; i++) {       // outer: n
        for (int j = 1; j < n; j *= 2) {  // inner: log n
            arr[i] += arr[j];
        }
    }
}
```

Outer loop: n iterations. Inner loop: j doubles each step so it runs log n times per outer iteration.
Multiply: n × log n = O(n log n).